PWU-AB-2 NOR flash probe — MTD availability + rootfs rebuild status

Revision: 1 **Last modified:** 2026-06-19T00:30:00Z

1. NOR flash probe result (CRITICAL FINDING)

Date: 2026-06-19 **Method:** Booted the existing rootfs.ext2 + kernel with `-accel tcg` (not HVF, to avoid the serial issue), logged in, and probed for MTD devices.

Result: NO MTD devices in the guest. Definitively proven:

Probe	Result	Meaning
`ls /dev/mtd*`	`MTDEXIT=1`	No MTD device nodes exist
`cat /proc/devices | grep mtd`	`NO_MTD_CLASS`	MTD subsystem not compiled into kernel
`cat /proc/iomem | grep flash`	`NO_IOMEM_FLASH_MATCH`	No flash region in memory map
`dmesg | grep -iE "physmap|cfi|flash|mtd"`	`NO_DMESG_MTD`	No MTD driver probed at all
`/dev/mem` access	`MEMEXIT=0`	`/dev/mem` IS accessible

Root cause: The `qemu_arm64_defconfig` Linux kernel does not include MTD drivers (CONFIG_MTD, CONFIG_MTD_CFI, CONFIG_MTD_PHYSMAP are not set). The QEMU `-machine virt` platform exposes the system flash/pflash at a fixed address in the memory map, but without kernel MTD support, the guest cannot see it as `/dev/mtd0`.

2. Impact on fw_env.config

Since the guest has NO MTD devices, the classic `fw_env.config` raw device form (`/dev/mtd0 <offset> <size>`) **cannot work**. Even if we add kernel MTD drivers, pflash appears at a specific memory address that needs `CONFIG_MTD_PHYSMAP` with the correct `physmap.addr` and `physmap.len` parameters.

`/dev/mem` IS accessible (zero-copy read of physical memory), so a userspace helper could potentially read/write the U-Boot env at the pflash base address directly. But `fw_setenv`/`fw_printenv` from `u-boot-tools` need `/dev/mtd*` or a working `fw_env.config` — neither is available.

Decision for now: The `fw_env.config` in the overlay stays COMMENTED OUT. The env mechanism is deferred to a follow-up PWU (see §5 below).

3. Rootfs rebuild (build_image.sh changes)

The build is running with these changes:

Config additions

Config symbol	Purpose
`BR2_PACKAGE_UBOOT_TOOLS=y`	Provides `fw_setenv`/`fw_printenv` in the guest
`BR2_PACKAGE_LVM2=y`	Provides `dmsetup` for dm-verity status probing
`BR2_ROOTFS_OVERLAY="/work/rootfs-overlay"`	Installs RAUC config files into the rootfs
`BR2_ROOTFS_OVERLAY_DELETE_STALE=y`	Ensures overlay replaces stale files

Rootfs overlay files

File in overlay	Status
`/etc/rauc/system.conf`	RAUC A/B slot config — **WIRED**
`/etc/fw_env.config`	**COMMENTED OUT** (no MTD — see §1)
`/etc/rauc/dev.cert.pem`	Dev cert for RAUC keyring — **WIRED** (generated inside container)

Key management improvement

The dev key+cert are generated INSIDE the container using `openssl req`, ensuring the cert in the rootfs keyring and the key extracted for host-side bundle signing are always in sync. The private key is: - Generated in the container at build time (ephemeral) - Copied to `/work/out/rauc-keys/dev.key.pem` inside the container - Extracted to `./out/rauc-keys/dev.key.pem` via `podman cp` - DELETED from the overlay BEFORE Buildroot copies it (so it NEVER ends up in the rootfs, per §11.4.10) - `chmod 600` on the host after extraction

4. Build status

Phase	Status
build_image.sh	**RUNNING** (started ~2026-06-19T00:28:00Z, ~40 min expected)
After build: boot_smoke.sh	PENDING

Phase	Status
After build: ab_rauc_verity.sh RED_MODE=1	PENDING

5. Remaining blockers for GREEN RED_MODE=0

#	Blocker	Status	Required action
1	`/etc/rauc/system.conf` in rootfs	**FIXED** (this build)	N/A
2	`dmsetup` in rootfs (lvm2)	**FIXED** (this build)	N/A
3	`fw_setenv`/`fw_printenv` in rootfs	**FIXED** (this build, uboot-tools)	N/A
4	`/etc/fw_env.config` — env mechanism	**BLOCKED** (no MTD)	Need alternative approach (see §5.1)
5	RAUC bundle (`update.raucb`)	**BLOCKED** (needs Linux)	Run `rauc bundle` on Linux host/CI
6	RAUC U-Boot backend env scheme reconciliation	**UNVERIFIED**	Align RAUC's `BOOT_ORDER+Boot_LEFT` with this project's `bootcount+upgrade_available` scheme

5.1 Alternative approaches for U-Boot env access

Since the kernel has no MTD and adding MTD drivers to the `qemu_arm64_defconfig` kernel requires a kernel rebuild, here are the viable paths:

PATH A (Recommended): Rebuild U-Boot with `CONFIG_ENV_IS_IN_FAT` - Change the U-Boot config from `CONFIG_ENV_IS_IN_FLASH` to `CONFIG_ENV_IS_IN_FAT` - The env is stored as a file `uboot.env` on the FAT boot partition (vda1) - `fw_env.config` uses the FAT-file form: `/dev/vda1 0x0 0x4000 uboot.env` - Both U-Boot and Linux userspace (`fw_setenv`/`fw_printenv`) can access the same env - **Effort:** Medium (rebuild U-Boot with `BR2_TARGET_UBOOT_CUSTOM_CONFIG` to add `CONFIG_ENV_IS_IN_FAT=y`) - **Risk:** Low (well-documented, used by Mender and others)

PATH B: Keep FLASH env, add kernel MTD drivers - Add `CONFIG_MTD=y`, `CONFIG_MTD_CFI=y`, `CONFIG_MTD_PHYSMAP=y` to the kernel - Determine the pflash base address (`-machine virt` exposes it at `0x0` with `-bios`) - This exposes the env region as `/dev/mtd0` - **Effort:** Medium (kernel rebuild, need correct physmap parameters) - **Risk:** Medium (kernel may not detect the CFI flash correctly under virt)

PATH C: Userspace `/dev/mem` helper - Write a simple helper that reads/writes the U-Boot env at the known pflash memory address via `/dev/mem` - `/dev/mem` is accessible (proven above) - **Effort:** High (need to implement the env format parser) - **Risk:** High (fragile, address-dependent, bypasses kernel abstractions)

PATH D (for initial GREEN only): Skip fw_setenv in test, set BOOT_ORDER from U-Boot prompt - The expect driver already sets BOOT_ORDER at the U-Boot prompt (line 198 in ab_rauc_verity.sh) - RAUC's `rauc install` needs fw_setenv to arm the boot selector - If we skip `rauc install`'s automatic slot arm and manually set BOOT_ORDER from U-Boot, we can still prove the dm-verity slot switch - **Effort:** Low (modify test driver) - **Compromise:** Proves dm-verity and slot switch but not the full RAUC apply->activate pipeline